Universidad de los Andes
Facultad de Ingeniería
Arquitectura de computadores – ICC3202
Segundo semestre 2026

Proyecto 2

Grupo 18

Cristóbal Aguirre, Ariam Astudillo, Antonio Bouchon

Este proyecto consiste en implementar un juego de reflejos en una FPGA, utilizando Pochoco SoC y el procesador Espino Core. En cada ronda, el sistema enciende los cuatro LEDs durante al menos tres segundos y luego selecciona uno como objetivo. El jugador debe presionar el botón asociado lo más rápido posible. Si acierta, el sistema calcula y muestra su tiempo de respuesta en décimas de segundo, si se equivoca, indica el error y repite la ronda sin perder los aciertos anteriores. Al completar diez respuestas correctas, muestra el tiempo promedio.

El juego se programa en assembly y se traduce a código máquina mediante un assembler. El diseño integra el procesador, la memoria y los periféricos necesarios para controlar LEDs, botones y displays, además de un contador de ciclos para medir el tiempo. En este informe se presentarán la arquitectura del sistema, la lógica del juego, la medición y el cálculo del promedio, el funcionamiento del assembler y las pruebas realizadas.

1. Arquitectura del juego

El sistema se implementa sobre una FPGA Lattice iCE40 HX1K en la placa de desarrollo Nandland Go Board, operando a una frecuencia de reloj base de 25 MHz. La arquitectura general se basa en el Pochoco SoC, una plataforma modular construida alrededor del procesador Espino Core, complementada con una memoria RAM unificada y un controlador de periféricos mapeados en memoria (MMIO) diseñado a la medida para controlar las entradas y salidas, generar temporizaciones precisas y capturar los tiempos de reacción del jugador a nivel de ciclo de reloj.

1.1. Estructura general del sistema y módulo top-level

El punto de partida del diseño es el SoC base, el cual integra el núcleo procesador, la memoria principal y el subsistema de periféricos. Para sintetizar y programar la solución en la Go Board, se desarrolló un módulo superior propio denominado `game_top.v`. Este módulo actúa como el *top-level* que se instancia en la FPGA, cumpliendo las siguientes funciones fundamentales:

- **Instanciación del SoC e inicialización de memoria:** Instancia a `pochoco_soc` configurando el parámetro `MemFile` con la ruta `"sw/game.hex"`. De esta manera, la memoria RAM unificada de 512 palabras de 32 bits (2048 bytes), sintetizada mediante las primitivas de bloque `SB_RAM40_4K` del iCE40, se inicializa con el código máquina del juego producido por el assembler.
- **Conexión con los periféricos físicos de la Go Board:** Conecta las señales de reloj de 25 MHz (`i_Clk`), los cuatro LEDs de usuario (`o_LED`), los cuatro botones pulsadores (`i_Switch`) y los catorce cátodos de los dos displays de siete segmentos (`o_Segment1` y `o_Segment2`).
- **Optimización de recursos lógicos:** La FPGA Lattice iCE40 HX1K dispone de recursos limitados (1280 celdas lógicas, cada una con una LUT de cuatro entradas y almacenamiento asociado). Para optimizar el uso del área y asegurar el cierre de temporización, `game_top` desactiva el controlador SPI mediante el parámetro `.EnableSPI(0)`, lo que ahorra una fracción sustancial de celdas lógicas manteniendo intacta la interfaz del SoC.
- **Restricciones de pines:** En el archivo de restricciones físicas `goboard.pcf` se fijaron las conexiones a los pines de la placa y se eliminaron las señales de la UART original, evitando advertencias de síntesis por puertos no utilizados.

El procesador Espino Core implementa la arquitectura RISC-V RV32E, contando con 16 registros arquitectónicos (`x0` a `x15`). Con el objetivo de economizar recursos en la FPGA, su unidad aritmético-lógica (ALU) mantiene desactivadas las operaciones de desplazamiento (*shifts*) y no incluye multiplicador ni divisor por hardware. Por ello, toda la lógica del juego, las esperas, la selección de objetivos y el procesamiento de tiempos se resuelven utilizando exclusivamente sumas, restas y operaciones lógicas bit a bit.

1.2. Modificaciones al RTL y mapa de memoria

Para cumplir con los requisitos del proyecto, se modificó el módulo de periféricos `pochoco_periph.v`. En el diseño original, este bloque permitía escrituras a los displays y a los LEDs, además de lecturas no sincronizadas de los botones. En la nueva implementación se agregaron un contador libre de ciclos de reloj, sincronizadores de entrada y un circuito de captura atómica de tiempo en hardware.

La comunicación entre el procesador y los periféricos se realiza mediante entradas y salidas mapeadas en memoria (MMIO), comenzando en la dirección base `0x80000000`. El decodificador del SoC detecta un acceso a periféricos cuando la dirección tiene su bit más significativo en alto (`data_addr[31] = 1`) y el bit 16 en bajo (`data_addr[16] = 0`). Todas las transacciones se efectúan en palabras alineadas de 32 bits, utilizando los bits `addr_i[7:2]` como desplazamiento interno.

En la Tabla 1 se presenta el mapa de memoria de los registros del periférico.

Offset	Registro / Función	Acceso	Descripción
+0x00	digit_q	Escritura	Escribe los dos dígitos en hexadecimal o BCD empaquetado (wdata[7:0]). Un decodificador interno traduce cada cuarteto a siete segmentos para los dos displays.
+0x04	led_q / Armado	Escritura	Escribe la máscara de los 4 LEDs (wdata[3:0]) y captura cycles_q en start_q. Si la máscara es unitaria (1, 2, 4 u 8), arma la medición (armed_q = 1); cualquier otro valor la desarma. Toda escritura de LEDs limpia valid_q y captured_q.
+0x08	btn_sync	Lectura	Retorna el estado sincronizado de los 4 botones (rdata[3:0]).
+0x0C	cycles_q	Lectura	Retorna el valor actual del contador libre de ciclos de 32 bits.
+0x10	start_q	Lectura	Retorna la marca temporal en ciclos de reloj de la última escritura sobre el registro de LEDs.
+0x14	elapsed_q	Lectura	Retorna los ciclos transcurridos de respuesta (cycles_q − start_q), congelados por hardware al registrarse la primera pulsación.
+0x18	Estado y captura	Lectura	Retorna en el bit 4 la bandera de captura válida (valid_q) y en los bits 3..0 la máscara del botón presionado (captured_q).

Cuadro 1: Mapa de memoria de los registros del periférico (pochoco_periph).

1.3. Contador de ciclos y sincronización de entradas

El cómputo del tiempo se fundamenta en el contador libre cycles_q, implementado en pochoco_periph.v como un registro de 32 bits que incrementa en cada flanco positivo de reloj mientras el sistema se encuentra fuera de reset:

$$\text{cycles_q}[k+1] = \text{cycles_q}[k] + 1.$$

A la frecuencia de operación de la Go Board (25 MHz), el periodo necesario para que este contador complete su recorrido y vuelva a cero es:

$$T_{\text{desborde}} = \frac{2^{32}}{25\,000\,000\,\text{Hz}} = \frac{4\,294\,967\,296}{25\,000\,000}\,\text{s} \approx 171{,}8\,\text{s}.$$

La resta modular sin signo de 32 bits permite obtener la diferencia de ciclos incluso si el contador cruza por cero, siempre que cada intervalo medido sea menor que 2^{32} ciclos, aproximadamente 171,8 s. El diseño no detecta intervalos que superen este límite. Los tiempos habituales de reacción son considerablemente menores, pero esta condición debe reconocerse como una limitación de la medición.

Por otra parte, los cuatro botones mecánicos de la Go Board generan señales asíncronas y activas en alto. Conectar estas entradas directamente a la lógica síncrona podría ocasionar problemas de metaestabilidad. Para reducir la probabilidad de propagación de metaestabilidad, el periférico incorpora dos biestables en cascada por cada botón:

$$\text{btn_meta}[k+1] \Leftarrow \text{btn_i}[k],$$
$$\text{btn_sync}[k+1] \Leftarrow \text{btn_meta}[k].$$

El sincronizador introduce una latencia de pocos ciclos, cuya relación con el instante físico de pulsación depende de la fase de llegada de la entrada asíncrona. Además, la lógica de captura utiliza el valor previamente almacenado en `btn_sync`. Esta latencia es despreciable respecto de una décima de segundo. La sincronización no elimina por completo el riesgo de metaestabilidad ni constituye por sí sola un filtro de rebote mecánico; el software implementa adicionalmente un filtro de liberación.

1.4. Temporización de tres segundos y captura atómica de respuesta

El desarrollo de cada ronda combina el control por software con la precisión de captura por hardware:

1. **Espera inicial de tres segundos:** Al inicio de la ronda, el programa escribe el valor 15 (`0b1111`) en `+0x04`, encendiendo los cuatro LEDs. Al no ser una máscara unitaria, la captura permanece desarmada (`armed_q = 0`). El procesador lee el instante de encendido en `start_q` (`+0x10`) y entra en un bucle que consulta repetidamente `cycles_q` (`+0x0C`). Mediante restas sin signo compara la diferencia frente a la constante `WAIT_CYCLES` (75 000 000 ciclos). La instrucción `bltu` retiene la ejecución hasta completar el tiempo reglamentario.

2. **Filtro de liberación de botones:** Finalizada la espera de tres segundos, el software invoca la rutina `release`, la cual verifica que los cuatro botones se mantengan sin presionar durante 500 000 ciclos continuos (20 ms). Esto evita que el jugador anticipe la jugada manteniendo un botón presionado o que se procesen rebotes mecánicos residuales de rondas previas.

3. **Inicio de ronda y armado atómico:** Una vez elegido el LED objetivo, el programa escribe su máscara ($1, 2, 4$ u 8) en `+0x04`. En ese mismo flanco de reloj se enciende el LED correspondiente, el hardware guarda el valor actual de `cycles_q` en `start_q`, pone `valid_q <= 0` y activa `armed_q <= 1`. El inicio de la ronda y la toma del tiempo inicial coinciden en el mismo instante lógico.

4. **Captura atómica en hardware:** Estando el sistema armado, en el primer flanco de reloj en que el sincronizador detecta una pulsación (`|btn_sync == 1`), el hardware congela de inmediato la diferencia:
$$\text{elapsed_q} \Leftarrow \text{cycles_q} - \text{start_q},$$
registra los botones pulsados en `captured_q`, levanta la bandera `valid_q <= 1` y desarma el circuito (`armed_q <= 0`).

Esta captura atómica en hardware desacopla por completo la medición del sondeo por software. El programa no necesita contar ciclos dentro de un bucle dependiente de la latencia de las instrucciones de la CPU, sino que simplemente consulta el bit `valid_q` y lee el valor en ciclos almacenado en `elapsed_q`.

1.5. Selección pseudoaleatoria del LED objetivo

El enunciado exige que la selección del LED objetivo sea perceptiblemente pseudoaleatoria y no repetitiva entre partidas. Ante la ausencia de operaciones de desplazamiento, multiplicación o división en la ALU de Espino Core, se implementó un generador pseudoaleatorio basado en mezcla de estado y muestras del contador en software, aprovechando la asincronía del jugador como fuente de entropía:

- **Muestreo de entropía humana:** Tras la espera de tres segundos y la confirmación de liberación de botones, el programa consulta el contador libre `cycles_q`. Como las fracciones de microsegundo en las que el usuario interactúa con la placa son variables e impredecibles, los bits bajos del contador permiten introducir variación temporal en la selección; no se realizó una caracterización estadística de su distribución.
- **Aislamiento y reducción modular:** El software extrae los bits 0 a 10 aplicando una máscara lógica (cycles_q & 2047). Para plegar estos bits sin utilizar instrucciones de división, el programa calcula el residuo $r = (\text{cycles_q} \;\&\; 2047) \pmod{31}$ mediante restas sucesivas de 31.
- **Actualización del estado del generador:** El valor r se mezcla con el registro de estado persistente (`x5`, con valor inicial 37) mediante una operación XOR y la suma de una constante:

$$\text{estado}_{k+1} = (\text{estado}_k \oplus r) + 73.$$

- **Conversión a máscara de LED:** El programa extrae los dos bits inferiores del estado (estado & 3), obteniendo un índice entre 0 y 3. Para transformar este índice en una máscara unitaria ($1, 2, 4$ u 8) sin instrucciones de desplazamiento, se inicializa un registro en 1 y se duplica iterativamente mediante sumas sucesivas (add $x7, x7, x7$).

La selección depende del estado acumulado y de muestras del contador afectadas por los tiempos de interacción del jugador. Esto permite obtener secuencias variables entre partidas, aunque pueden repetirse objetivos consecutivos y no se garantiza una distribución uniforme. Una ejecución con los mismos tiempos de entrada puede reproducir la secuencia; el primer objetivo después del encendido también puede ser reproducible. No se trata de un generador criptográfico.

1.6. Inicialización y corrección del arranque

Se amplió el contador de reset de encendido de `pochoco_soc.v` a 16 bits, manteniendo el procesador en reset durante 65535 ciclos. A 25 MHz, este intervalo corresponde a:

$$T_{\text{reset}} = \frac{65535}{25\,000\,000}\,\text{s} \approx 2{,}62\,\text{ms}.$$

Además, se modificó `espino_register_file.v` para escribir explícitamente cero en `x0` de ambos bancos durante el reset. En ejecución normal, las escrituras del programa a este registro se bloquean.

La aplicación conjunta de ambas modificaciones permitió el arranque correcto del juego en la Go Board. No se realizaron pruebas físicas separadas que permitan determinar cuál de los dos cambios era suficiente por sí solo. El reset inicializa físicamente `x0`; los demás registros necesarios se preparan desde el programa antes de utilizarlos.

2. Assembly

El programa del juego está escrito en el archivo sw/game.s, y se ejecuta en el procesador base proporcionado Espino Core. Este se comunica con los periféricos mediante direcciones de memoria mapeadas. El registro x2 contiene la dirección base 0x80000000, a partir de ella, el programa accede a los LEDs, botones, display y contador de ciclos.
Los registros principales utilizados se resumen a continuación:

Registro	**Función**
`x0`	Valor cero; se descartan las escrituras de instrucciones.
`x1`	Dirección de retorno de las subrutinas.
`x2`	Dirección base de los periféricos.
`x3`	Cantidad de respuestas correctas.
`x4`	Suma de tiempos cuantizados en décimas.
`x5`	Estado de la selección pseudoaleatoria.
`x6-x14`	Temporales para máscaras, timestamps, límites y conversión.

Cuadro 2: Uso de los registros del programa.

Las instrucciones efectivamente utilizadas en `game.s` son `lui`, `addi`, `add`, `sub`, `xor`, `andi`, `lw`, `sw`, `beq`, `bne`, `bltu`, `jal` y `jalr`, además de la pseudoinstrucción `li`.

Las llamadas usan `jal x1, etiqueta` y retornan con `jalr x0, 0(x1)`. Las subrutinas no realizan llamadas anidadas, por lo que no requieren preservar un segundo enlace ni utilizar una pila. El registro `x2` se usa como base MMIO, no como puntero de pila.

El flujo del juego se organiza mediante etiquetas. `round` inicia una ronda, `wait_press` espera una respuesta, `error` muestra el error y reinicia la ronda actual y `display` convierte el resultado para mostrarlo en los displays. Las rutinas pause y release implementan esperas temporizadas y verifican que los botones estén sueltos antes de continuar.

Al comenzar cada ronda, el programa enciende los cuatro LEDs y espera el tiempo inicial. Luego, obtiene una "semilla"del contador de ciclos, selecciona uno de los cuatro LEDs y espera una pulsación. Si el botón coincide con el LED objetivo, registra el tiempo de respuesta, incrementa el contador de aciertos y muestra el tiempo. Si la pulsación es incorrecta, muestra EE y vuelve a intentar la ronda sin modificar los aciertos ni la suma acumulada. Después de diez aciertos, calcula y muestra el promedio.

2.1. Fragmento de espera inicial

El siguiente fragmento del código enciende todos los LEDs, recupera el timestamp de esa escritura y espera la diferencia de ciclos requerida:

Listing 1: Espera de al menos tres segundos.

```
round:
    addi x6, x0, 15
    sw x6, 4(x2)        # LEDs e inicio registrados juntos
    lw x7, 16(x2)      # timestamp inicial
    li x8, WAIT_CYCLES
wait_three:
    lw x9, 12(x2)      # contador actual
    sub x9, x9, x7     # diferencia modular
    bltu x9, x8, wait_three
    jal x1, release
```

La comparación sin signo `bltu` mantiene la espera mientras la diferencia sea menor que el umbral. La llamada posterior exige una liberación estable de los botones.

2.2. Medición del tiempo de respuesta

El reloj de la Go Board opera a una frecuencia de 25 MHz. Por lo tanto, la duración de un ciclo es

$$T_{\text{ciclo}} = \frac{1}{25\,000\,000}\,\text{s} = 40\,\text{ns}.$$

Para mantener encendidos los cuatro LED durante al menos tres segundos, el programa utiliza la constante `WAIT_CYCLES`, definida como 75 000 000 ciclos:

$$N_{\text{espera}} = 3\,\text{s} \cdot 25\,000\,000\,\frac{\text{ciclos}}{\text{s}} = 75\,000\,000\,\text{ciclos}.$$

Durante esta espera, el programa consulta el contador de ciclos y compara el valor actual con el valor registrado al inicio. La espera termina cuando la diferencia alcanza el umbral establecido. Debido a que el programa consulta el contador mediante instrucciones de lectura y comparación, el intervalo efectivo puede ser ligeramente superior a tres segundos.

La medición del tiempo de reacción comienza cuando el programa escribe la máscara del LED objetivo. En ese mismo instante, el periférico guarda el valor actual del contador de ciclos. Cuando detecta una pulsación, registra el tiempo transcurrido desde ese instante y la máscara del botón presionado. Así, el tiempo de respuesta se obtiene mediante el contador de ciclos, sin depender de la duración del ciclo de instrucciones que toma el software para procesar la pulsación.

Para expresar el resultado en décimas de segundo, se utiliza la constante `TENTH_CYCLES`, equivalente a 2 500 000 ciclos:

$$N_{\text{décima}} = 0{,}1\,\text{s} \cdot 25\,000\,000\,\frac{\text{ciclos}}{\text{s}} = 2\,500\,000\,\text{ciclos}.$$

Sea c_i el tiempo de respuesta de la ronda i, expresado en ciclos. El tiempo mostrado en décimas, d_i, se calcula como

$$d_i = \left\lfloor \frac{c_i}{2\,500\,000} \right\rfloor .$$

El programa implementa esta conversión mediante restas sucesivas de `TENTH_CYCLES`. Cada resta completa representa una décima de segundo. Por ejemplo, un resultado de 12 corresponde a 1,2 s. La conversión trunca las fracciones de décima restantes.

El bucle de conversión utiliza `x9` para los ciclos restantes y `x10` para contar las décimas completas:

Listing 2: Conversión mediante restas sucesivas.

```
    addi x10, x0, 0
    li x8, TENTH_CYCLES
convert:
    bltu x9, x8, converted
    sub x9, x9, x8
    addi x10, x10, 1
    jal x0, convert
converted:
    add x4, x4, x10
    addi x3, x3, 1
```

2.3. Cálculo del promedio de diez rondas

El registro `x3` almacena la cantidad de respuestas correctas y el registro `x4` acumula sus tiempos, expresados en décimas de segundo. Para cada respuesta correcta, el programa suma d_i a `x4` e incrementa `x3`. Una pulsación incorrecta reinicia la ronda actual, pero no modifica estos registros, por ello, se conservan los aciertos anteriores.

Al completar diez respuestas correctas, el programa calcula el promedio de los tiempos cuantizados:

$$\overline{d} = \left\lfloor \frac{\displaystyle\sum_{i=1}^{10} d_i}{10} \right\rfloor .$$

El cálculo se realiza mediante restas sucesivas de diez sobre la suma acumulada. El cociente obtenido es el promedio en décimas de segundo y se muestra en los displays con dos dígitos. Por ejemplo, un promedio de 15 se presenta como 15, que representa 1,5 s.

Tanto la conversión de cada tiempo como el cálculo del promedio descartan la parte fraccionaria mediante truncamiento. Además, la rutina de visualización limita el valor mostrado a 99; esta saturación afecta la representación en los displays, no la suma utilizada internamente para calcular el promedio.

Listing 3: Promedio entero de los tiempos cuantizados.

```
    addi x6, x0, ROUNDS
    # Tras confirmar que x3 alcanzo ROUNDS:
    addi x10, x0, 0
average:
    bltu x4, x6, final
    sub x4, x4, x6
    addi x10, x10, 1
    jal x0, average
final:
    jal x1, display
```

El fragmento anterior muestra las instrucciones relevantes del promedio; en el código completo también está la comparación que decide si comenzar otra ronda. La división modifica `x4`, porque la suma ya no se utiliza después de calcular el resultado final. Tras mostrar el promedio, el programa espera una pulsación y su liberación para reiniciar la partida.

2.4. Visualización decimal y manejo de errores

El periférico decodifica los dos cuartetos de `digit_q` como dígitos hexadecimales. Para mostrar un resultado decimal, la rutina `display` separa decenas y unidades mediante restas de diez y construye BCD empaquetado. Por ejemplo, el número decimal 23 se escribe como `0x23`; escribir directamente 23, equivalente a `0x17`, mostraría 17.

Cada decena agrega 16 al cuarteto superior, y el resto representa las unidades. La saturación visual en 99 corresponde a 9,9 s, pero no altera el tiempo previamente sumado en `x4`.

Una máscara de botones distinta de la del objetivo, incluyendo pulsaciones múltiples, activa la ruta `error`. Esta escribe `0xEE`, apaga los LEDs y repite la ronda después de una espera de un segundo y de la liberación estable, sin cambiar `x3` ni `x4`.

3. Assembler

3.1. Assembler: codificación, etiquetas, validación y pruebas

El assembler desarrollado para el proyecto se encuentra en `tools/assembler.py`. Su función es traducir el programa escrito en `sw/game.s` a instrucciones de máquina de 32 bits, que se almacenan en `sw/game.hex`, una palabra hexadecimal por línea. El assembler está implementado en Python y realiza esta traducción sin invocar un assembler externo.

3.1.1. Sintaxis e instrucciones soportadas

El assembler acepta registros `x0` a `x15`, inmediatos decimales, hexadecimales y negativos, comentarios iniciados con `#`, etiquetas y constantes definidas mediante `.equ`. Las instrucciones implementadas se agrupan de la siguiente manera:

- Tipo R: `add`, `sub`, `xor`, `or`, `and`, `slt` y `sltu`.
- Tipo I: `addi`, `xori`, `ori`, `andi`, `slti` y `sltiu`.
- Acceso a memoria: `lw` y `sw`.
- Saltos y branches: `jal`, `jalr`, `beq`, `bne`, `blt`, `bge`, `bltu` y `bgeu`.
- Carga de inmediatos: `lui` y `li`.

3.1.2. Codificación de instrucciones

El assembler separa cada línea en la operación y sus operandos. Después, identifica el formato de la instrucción y coloca los campos correspondientes en sus posiciones dentro de una palabra de 32 bits. Se utilizan los formatos R, I, S, B, U y J, de acuerdo con el tipo de operación.

Por ejemplo, la instrucción

```
lw x9, 12(x2)
```

indica que se debe leer una palabra desde la dirección contenida en `x2` más el desplazamiento 12, y guardar el resultado en `x9`. Su codificación es

`00c12483`.

En este ejemplo de tipo I, los campos se combinan como:

$$(12 \ll 20) \mid (2 \ll 15) \mid (2 \ll 12) \mid (9 \ll 7) \mid 3.$$

El inmediato es 12, `rs1` es 2, `funct3` es 2, `rd` es 9 y el opcode es 3. Los desplazamientos colocan cada campo en su posición y el OR combina los bits. El resultado es `0x00C12483`.

La instrucción `li` se expande siempre a dos instrucciones: `lui` y `addi`. Así se pueden cargar valores que no caben en un inmediato de 12 bits. La expansión tiene un tamaño fijo de ocho bytes, lo que permite calcular correctamente las direcciones de las instrucciones durante la resolución de etiquetas.

3.1.3. Ejemplo de traducción ejecutable

El archivo `tools/ejemplo_traduccion.py` utiliza el assembler del proyecto para traducir `lw x9, 12(x2)` y mostrar sus campos y su representación binaria y hexadecimal en la consola:

```
python -B .\tools\ejemplo_traduccion.py
```

La opción `-B` evita generar archivos de caché de Python. El ejemplo no modifica la imagen de memoria del juego. Para esta instrucción de formato I se obtiene:

Campo	Posición	Valor decimal
Inmediato	Bits 31–20	12
`rs1`	Bits 19–15	2
`funct3`	Bits 14–12	2
`rd`	Bits 11–7	9
Opcode	Bits 6–0	3

Cuadro 3: Campos de la instrucción utilizada en el ejemplo ejecutable.

Los campos concatenados producen el resultado:

```
Instruccion: lw x9, 12(x2)
Binario: 00000000110000010010010010000011
Hexadecimal: 00c12483
```

En el juego, `x2` contiene `0x80000000`; por lo tanto, esta carga accede al contador de ciclos en `0x8000000C`.

3.1.4. Resolución de etiquetas

El proceso de ensamblado se realiza en dos pasadas:

1. En la primera pasada se procesan las constantes declaradas con `.equ`, se registran las etiquetas y se calcula la dirección de cada instrucción. Para este cálculo, las instrucciones normales ocupan cuatro bytes y cada instrucción `li` ocupa ocho bytes.

2. En la segunda pasada se codifica cada instrucción. Las referencias a etiquetas se reemplazan por desplazamientos relativos a la dirección de la instrucción que realiza el salto o branch.

Por ejemplo, una instrucción branch que hace referencia a una etiqueta debe codificar la diferencia entre la dirección de destino y la dirección de la propia instrucción. En cambio, cuando se proporciona un desplazamiento numérico, este se interpreta directamente como un desplazamiento relativo.

3.1.5. Validación de entrada

Antes de generar el código máquina, el assembler verifica que las instrucciones estén soportadas y que cada una reciba la cantidad correcta de operandos. También comprueba que los registros estén

en el rango x0–x15, que las etiquetas no se repitan y que las referencias a etiquetas y constantes puedan resolverse.

Los inmediatos se validan según el formato de la instrucción. Por ejemplo, los inmediatos de 12 bits deben estar dentro del rango con signo correspondiente, mientras que lui acepta un inmediato sin signo de 20 bits. Para branches y saltos también se revisan el rango del desplazamiento y la alineación del destino. Finalmente, se rechazan los programas que superen las 512 palabras previstas para la memoria.

Ante errores de codificación de instrucciones, el assembler informa el número de línea y el tipo de error; la comprobación global de capacidad de memoria informa el exceso de palabras. Si el ensamblado termina correctamente, escribe las palabras de 32 bits en hexadecimal en game.hex y genera además game.symbols.json, que contiene las direcciones de las etiquetas y los valores de las constantes.

3.1.6. Pruebas del assembler

Las pruebas automatizadas están en tests/test_assembler.py. Comprueban la codificación de instrucciones conocidas, la resolución de etiquetas, la expansión de li y el rechazo de entradas inválidas. Para ejecutar las pruebas incluidas en el proyecto se utiliza:

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\build.ps1 test
```

3.1.7. Casos y resultados concretos del assembler

Se ejecutaron los cuatro métodos de prueba de tests/test_assembler.py, que agrupan varios casos. La ejecución finalizó correctamente:

```
Ran 4 tests
OK
```

Caso	Resultado esperado	Resultado observado
Codificación de lw x9,12(x2)	00c12483	Coincide al ensamblar la instrucción.
li x1,2048	000010b7 y 80008093	Correcto en el test de expansión.
Salto hacia etiqueta anterior	Desplazamiento negativo relativo al PC	Correcto en el test de etiquetas.
Registro x16	Rechazo	ValueError.
Inmediato 2048 en addi	Rechazo por rango de 12 bits	ValueError.
Etiqueta desconocida o duplicada	Rechazo	ValueError.
Shift o destino desalineado	Rechazo	ValueError.

Cuadro 4: Ejemplos de codificación y validación del assembler.

La codificación de lw x9,12(x2) se contrastó además directamente con el assembler. Estos resultados verifican los casos descritos; no constituyen una prueba exhaustiva de todos los programas RISC-V posibles.

4. Verificación del sistema y funcionamiento en FPGA

4.1. Simulación del SoC

El script `tests/simulate.py` utiliza Icarus Verilog y el modelo de primitivas `SB_RAM40_4K` provisto con Yosys. Para acelerar la ejecución, ensambla una copia del programa con constantes temporales reducidas en `build/sim.hex`, sin modificar la imagen `sw/game.hex` destinada a la placa.

El test bench `tests/tb_game.v` genera un reloj de 25 MHz, estimula los botones y compara los resultados. En la simulación realizada durante el desarrollo se verificaron:

- Inicialización de `x0`: se imponen valores iniciales incorrectos en ambos bancos y se comprueba que el reset los sustituya por cero.
- Espera inicial: los cuatro LEDs permanecen encendidos durante el umbral de simulación.
- Error inicial y error después de tres aciertos: se muestra `EE` y se repite la ronda conservando el progreso.
- Diez respuestas correctas: se comprueba la conversión del tiempo, la visualización y el promedio esperado.
- Variación de objetivos y reinicio de partida después del promedio.

La ejecución terminó con el mensaje:

```
PASS: espera, error, 10 rondas, tiempos, promedio, variacion y reinicio
```

La simulación verifica el recorrido lógico con tiempos reducidos. No sustituye la comprobación física del intervalo de tres segundos ni garantiza el comportamiento ante todas las condiciones eléctricas de los botones.

4.2. Construcción y programación

Se utilizó OSS CAD Suite. Yosys sintetiza el RTL, NextPNR coloca y conecta el circuito para la iCE40 HX1K en encapsulado `vq100`, Icepack genera el bitstream e Iceprog programa y verifica la memoria Flash.

La instalación utilizada presentó un fallo interno de ABC9. El script de construcción emplea las etapas de síntesis de iCE40 con ABC clásico y LUTs de cuatro entradas, antes de generar el JSON para NextPNR. Los comandos se guardan en un archivo `.ys` para evitar problemas de comillas y rutas con espacios en Windows PowerShell.

Los comandos de reproducción, también documentados en el README, son:

```
# Regenerar codigo maquina desde game.s
powershell -NoProfile -ExecutionPolicy Bypass -File .\build.ps1 assemble

# Ejecutar pruebas del assembler y simulacion
powershell -NoProfile -ExecutionPolicy Bypass -File .\build.ps1 test

# Sintetizar, rutear y generar bitstream
powershell -NoProfile -ExecutionPolicy Bypass -File .\build.ps1 build

# Construir y programar la Go Board
powershell -NoProfile -ExecutionPolicy Bypass -File .\build.ps1 program
```

En la construcción verificada del diseño corregido se obtuvieron los siguientes resultados:

Métrica	**Resultado**
Celdas lógicas	1277 de 1280, aproximadamente 99,8 %
Bloques RAM	12 de 16, 75 %
Frecuencia objetivo	25 MHz, cumplida
Frecuencia máxima estimada tras ruteo	35,45 MHz
Tamaño del bitstream	32220 bytes
Tamaño del programa	99 palabras, 396 bytes

Cuadro 5: Resultados de la construcción verificada después de corregir el arranque.

La frecuencia máxima es una estimación de temporización de NextPNR, no una medición física. El margen de área es reducido; cualquier extensión del RTL debe volver a sintetizarse y rutearse.

4.3. Comprobación en la Go Board

La programación de Flash finalizó con `VERIFY OK`, que confirma la coincidencia de los bytes escritos con el bitstream. Inicialmente, el juego permanecía sin actividad; una prueba independiente de displays, LEDs, botones y reloj funcionó correctamente. Después de aplicar las dos modificaciones de arranque descritas, se confirmó el funcionamiento del juego en la Go Board.

La confirmación funcional no debe confundirse con una medición instrumental de los tiempos. La demostración presencial contempla los cuatro LEDs durante al menos tres segundos, una respuesta correcta y su tiempo, diez aciertos y el promedio, y una pulsación incorrecta que repita la ronda actual.

5. Conclusiones y limitaciones

Se implementó un juego de reflejos en assembly sobre Pochoco SoC y Espino Core, con un assembler propio, contador de ciclos accesible por MMIO y captura de respuesta en hardware. La escritura del LED objetivo y el registro del inicio coinciden en un mismo flanco lógico. El software convierte los ciclos a décimas, conserva los aciertos frente a errores y calcula el promedio de diez tiempos cuantizados.

La simulación permitió verificar codificación, validación de entrada y recorrido lógico del juego. La corrección conjunta del reset y de la inicialización de `x0` permitió su funcionamiento en la placa. Permanecen como limitaciones la cuantización por truncamiento, la saturación visual en 99, el límite de aproximadamente 171,8 segundos por intervalo, el filtrado de liberación que no valida estabilidad de la primera pulsación y la selección pseudoaleatoria sin garantía de uniformidad. El diseño utiliza casi toda el área disponible de la HX1K.

Referencias

[1] Universidad de los Andes. *Proyecto 2: Juego en Assembly sobre Pochoco SoC*. Arquitectura de Computadores, semestre 2026-2. Enunciado del curso.

[2] Nicolás Villegas. *Pochoco SoC y Espino Core*. Repositorio base: https://github.com/nic0villegasc/pochoco_soc.

[3] Nandland. *The Go Board*. Características de placa y reloj: https://nandland.com/the-go-board/.

[4] RISC-V International. *RV32I Base Integer Instruction Set*. Referencia de registros y formatos: https://docs.riscv.org/reference/isa/v20260120/unpriv/rv32.html.